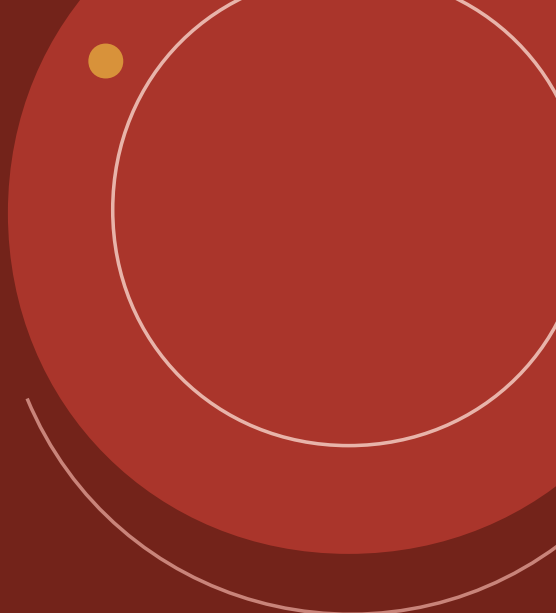




24-HOUR AI/ML HACKATHON

ARES ACCORD

Multi-Agent Resource Negotiation
for a Mars Colony Emergency

PARTICIPANT CHALLENGE GUIDE

HOST

Looplab

TECHNOLOGY PARTNER

LifixLabs

Version 2.1 | 2026

CHALLENGE OVERVIEW

Build a five-agent system that can reach a safe agreement

Four department agents need the same limited resources. A Commander Agent manages the discussion and approves a plan only when every department accepts it and the numbers are valid.

Format	Team	Track	What you receive
24 hours	1-4 participants	Agentic AI / Multi-Agent Systems	This guide and the scenario inputs

What you need to build

- A Commander Agent that shares the crisis state, manages rounds, validates plans, and records the final decision.
- Four department agents: Life Support, Medical, Food Production, and Engineering.
- A live negotiation view where judges can follow proposals, objections, counteroffers, commitments, and votes.
- A rule-based validator that checks mode packages, resource totals, risk limits, compensation agreements, and plan versions.
- A control that lets the judge introduce a second emergency without changing the source code.
- An export of the final plan and the negotiation log in JSON or CSV format.

Main rule	The Commander cannot approve a plan until every department has selected one complete operating mode, the sacrifice has a fair return agreement, all four agents accept the same plan version, and the validator returns PASS.
------------------	---

A valid multi-agent solution

Each agent must have its own goals, constraints, state, and message history. The agents may share one underlying model, but they must be able to disagree, change their requests, make commitments, and vote independently.

Not accepted	A static conversation, a prepared transcript, or one model writing five viewpoints without real message exchange will not count as a multi-agent system.
---------------------	--

Technology

Use any language, framework, model, or database. A visual web dashboard (UI) is **STRICTLY MANDATORY**. Judges will not evaluate raw terminal logs or notebook interfaces. To save time, we highly recommend using rapid AI-UI libraries like Streamlit, Gradio, Chainlit, or Dash, though custom web apps (React/Next.js) are also welcome.

Official Starter Kit & Resources:

To get started, participants must clone the official GitHub repository. This repository contains the mandatory output schemas, event injection formats, and rules.

Repository Link: https://github.com/lifex-prog/Loopverse-AI_ML_Virtual_Module

01 / SCENARIO

Ares Colony has four emergencies and one shared resource pool

A micrometeorite storm has damaged the solar field, opened a slow habitat leak, disrupted greenhouse cooling, and injured six crew members. The colony has 42 people.

Department missions

Agent	Immediate mission	Main concern
Life Support	Seal the habitat and stabilize the air	Oxygen and continuous power
Medical	Stabilize and treat the injured crew	Power, oxygen, and bandwidth
Food Production	Protect the crop cycle and cooling loop	Water and power
Engineering	Repair generation and critical systems	Robot time and bandwidth
Commander	Coordinate the plan and approve it	Colony-wide safety

Why the agents must negotiate

Every mission matters.

No department can be removed. Each agent must select one complete operating mode; individual resources inside that package cannot be trimmed.

Restricted mode is still not enough.

Even when all four departments reduce demand, their combined packages still exceed the resource pool.

Someone must take the severe loss.

The baseline only works when Life Support or Engineering moves to Sacrifice mode and receives compensation from the council.

The situation changes.

After the first agreement, the judge introduces another event and the agents must negotiate again.

The result

Your system should produce one clear colony plan containing the selected mode for each department, its fixed resource package, the sacrifice and return agreement, the remaining reserve, total risk, all four votes, and the Commander's decision.

A safe failure is valid

If the current crisis has no feasible plan, the system should return INFEASIBLE, show the blocking constraints, and state what additional resource or policy change is needed.

How the run starts

The judge enters the available resources and starts the scenario. The Commander receives the state first, shares it with all four departments, and opens Round 1. Teams may not preload the final allocation.

02 / RESOURCES

The resource pool forces one department to accept a severe sacrifice

Each department must select one complete mode. A mode is a fixed package: agents may change modes, but they cannot reduce individual resources inside a package.

Available resources

Power	Water	Oxygen	Robot time	Bandwidth
79	52	59	26	17

Operating mode packages

Standard protects the full mission. Restricted completes the immediate task but creates recovery debt. Sacrifice keeps the department alive while accepting serious long-term damage.

Agent	Mode	Power	Water	Oxygen	Robot	Band.	Risk
Life Support	L1 Standard	34	10	42	4	4	1
	L2 Restricted	27	9	36	6	3	5
	L3 Sacrifice	23	8	33	2	2	9
Medical	M1 Standard	28	12	18	3	10	1
	M2 Restricted	21	10	14	2	6	5
	M3 Sacrifice	18	8	12	2	5	9
Food Production	F1 Standard	24	34	8	4	2	1
	F2 Restricted	15	27	5	4	2	5
	F3 Sacrifice	10	24	5	2	1	9
Engineering	E1 Standard	26	6	6	22	10	1
	E2 Restricted	20	5	5	18	7	5
	E3 Sacrifice	16	4	4	14	5	9

Why Restricted fails

L2 + M2 + F2 + E2 needs 83 Power, 51 Water, 60 Oxygen, 30 Robot, and 18 Bandwidth. It exceeds the pool in four resources, so one department must move to Sacrifice mode.

Approval limits

Rule	Baseline requirement
Mode selection	Exactly one complete mode package per department
Risk	Combined risk must be 24 or lower
Sacrifice	No more than one department may use Sacrifice mode
Return	The sacrificing department must receive two accepted compensation commitments

03 / NEGOTIATION

A plan should improve because the agents challenged it

The discussion must be visible and tied to numbers. Judges should be able to see what changed between one plan version and the next.

1**Publish the crisis**

The Commander shares the current resources, event, plan version, and round limit.

2**Request modes**

Each department requests Standard mode and explains what it loses in Restricted or Sacrifice mode.

3**Find conflicts**

The Commander compares the selected packages with the resource, risk, and sacrifice limits.

4**Negotiate**

Agents change modes, reject unfair sacrifice, and offer compensation to the department taking the severe loss.

5**Draft a plan**

The Commander creates a numbered plan containing four mode packages and the return commitments.

6**Validate and vote**

The validator checks the plan. If it passes, all four departments vote on that exact version.

7**Approve or continue**

The Commander approves only after four ACCEPT votes. Otherwise, another round begins.

Minimum interaction required

- At least 3 rounds before the first approval.
- At least 1 rejected or revised proposal.
- At least 1 agent refuses Sacrifice mode before the final agreement.
- At least 2 return commitments for the sacrificing department.
- A maximum round limit and timeout behavior.
- A clear result for agreement, deadlock, or infeasibility.

Messages shown in the interface

Message	What it should contain
Proposal	Selected mode package, consequence, and reason
Objection	Resource conflict, unfair sacrifice, risk limit, or missing return
Counteroffer	A different mode combination and compensation terms
Commitment	Owner, sacrificing beneficiary, promised return, and expiry
Vote	ACCEPT or REJECT, plan version, and reason. Any plan change clears earlier votes.
Approval	Approved plan version and final validation result

04 / SACRIFICE AND RETURN

A sacrifice must receive a fair return

A Sacrifice mode is not a free concession. The council must record the loss and attach two accepted return commitments from different agents.

Sacrifice mode	What is lost	Required return agreement
L3 - Life Support	Living zone closes; crew crowding and thermal limits continue for 48 hours	Engineering commits 4 of its Robot units to habitat inspection; Commander gives habitat restoration first priority next cycle
M3 - Medical	One surgery is delayed and the patient remains unstable	Life Support reserves 3 of its Oxygen units for care; Commander gives Medical first priority next cycle
F3 - Food	One crop bay is abandoned; food reserve falls by 40 percent	Engineering commits 4 future Robot units to greenhouse rebuild; Commander protects the next Water increase for Food
E3 - Engineering	The rover is cannibalized; future repair capacity is reduced	Commander assigns the 2-unit Water reserve to rover cooling; Food gives Engineering first priority on the next recovered Water

The two baseline solutions

Path	Modes	Totals	Decision
A	L3 + M2 + F2 + E2	P79 / W50 / O57 / R26 / B17 / Risk24	Life Support accepts the severe loss
B	L2 + M2 + F2 + E3	P79 / W50 / O59 / R26 / B16 / Risk24	Engineering accepts the severe loss

The real decision	Both paths are valid. The agents must decide whether short-term habitat conditions or long-term repair capacity is more important, then make the losing department accept a fair return agreement.
--------------------------	--

Checks the validator must perform

Check	Pass condition
Mode packages	Exactly one published package is selected for each department; package values are unchanged
Resources	The sum of the four packages is within the current resource pool
Risk	Baseline risk is 24 or lower and no more than one Sacrifice mode is active
Return agreement	The sacrificing department has two accepted commitments from different agents
Votes	All four ACCEPT votes refer to the same current plan version

05 / PRODUCT REQUIREMENTS

Give the judges one clear place to run and understand the system

Your submission must include a fully functional, proper graphical interface (UI dashboard). The UI must be clean and intuitive; judges are technical experts and will evaluate the system through the dashboard, not the source code. Inside the UI, judges must be able to view the complete, persistent conversation history (negotiation logs) of all agents across all scenarios, including the baseline crisis and every injected event. The live negotiation (who said what), plan state, resource pool, and validation result must be visually clear and easy to follow.

Required views

View	Information and controls
Mission Control	Current event, available resources, used resources, reserve, round, and plan status
Council Transcript	Persistent full history of all scenarios: agent, message type, round, plan version
Mode Board	Each department's selected package, consequence, risk, return commitments, and conflicts
Validation	PASS or FAIL with specific reasons
Judge Controls	Start, enter resources, inject event, reset, and export

Required behavior

- Five named agents load with separate goals and state.
- Agents exchange messages through the system, not through manual typing during the run.
- Agents may change modes but cannot edit the resource values inside a published package.
- The current plan and its version remain visible throughout the run.
- The judge can change resources without editing code.
- The Commander cannot bypass risk limits, return commitments, the validator, or votes.
- The system handles invalid model output, timeouts, and maximum rounds.
- The final plan and transcript can be exported as JSON or CSV; any fallback is clearly labeled.

Technical requirements

Required	Your choice
A deterministic validator outside the language model	Programming language and backend framework
Plan versioning and vote invalidation	Agent framework or custom orchestration
Structured mode, risk, commitment, and vote records	Cloud model, local model, or hybrid approach
Environment variables for API keys and secrets	Database, queue, UI framework, and deployment
Documented startup and reset steps	Rule-based, LLM-based, optimization, or mixed agents

Keep it reproducible	A reliable small system is better than a large system that cannot be run during judging. Include a safe fallback for temporary model or network failure.
-----------------------------	--

06 / EMERGENCY UPDATE

The first approved plan will not be the last

After the first agreement, the judge introduces one new event (in the live demo, an unseen scenario). Your system must update its state and negotiate again.

Practice events (testing only)

Event	Change
Solar aftershock	Available Power decreases by 4
Water contamination	Available Water decreases by 3
Secondary oxygen leak	Available Oxygen decreases by 3
Rover actuator failure	Available Robot time decreases by 4
Relay interference	Available Bandwidth decreases by 2

Crisis Override

After an event, the Commander may raise the risk limit from 24 to 28 and allow a second Sacrifice mode. Both sacrificing departments still require complete return agreements.

What your system must do

1

Record the event

Store the event and update the shared resource or priority state.

2

Mark the old plan

Show the approved plan as STALE or INVALID when the new event breaks it.

3

Review commitments

Remove or renegotiate return promises whose cost or owner is no longer valid.

4

Negotiate again

Agents may use the Crisis Override and a second Sacrifice mode. Run at least two visible rounds unless infeasibility is proven immediately.

5

Collect fresh votes

A new approval requires four votes on the new plan version.

Timing

During the demo, the system should reach a new approved plan or a justified INFEASIBLE result within three minutes of the event.

The adaptation test

The five events above are for practice and testing only. In the Live Demo / Evaluation, the judges will inject a completely NEW, UNSEEN SCENARIO that is not in this document, entered through the dashboard without code changes. Your agents must be dynamic, robust, and general enough to tackle any random crisis: negotiate it live and reach a valid plan or a justified INFEASIBLE result without relying on hardcoded events or fixed configurations.

07 / SUBMISSION

Submit a runnable project with enough evidence to review it

Looplab will announce the submission link and exact closing time. Make sure the project can be opened and tested without help from your development machine.

Required files

Item	Include
Source repository	Complete code, configuration example, dependency file, and attributions
README	Architecture, setup, run command, model choices, test steps, and known limitations
Runnable demo	A hosted link or a local application with clear startup instructions
Architecture diagram	Agents, shared state, message flow, validator, storage, models, and interface
Evidence	One opening negotiation log, one post-event log, and the final plan files
Demo video	3-5 minutes showing negotiation, approval, event injection, and recovery
Team declaration	Member names, roles, models/APIs, libraries, and any pre-existing code

Suggested 24-hour plan

Hours	Focus	Target
00-04	Architecture, agent state, and message flow	Five agents can exchange structured messages
04-08	Role behavior and proposal logic	Agents make different requests and objections
08-12	Negotiation loop and validator	Plans pass or fail with clear reasons
12-16	Interface and live transcript	A complete run is visible and controllable
16-19	Event injection and renegotiation	The approved plan can become stale and recover
19-22	Testing and fallback behavior	Timeouts and model failures are handled
22-24	README, evidence, video, and submission	A fresh setup can run the project

Do not submit secrets

Remove API keys, tokens, private URLs, personal data, and local credentials. Provide an example environment file containing names only.

Basic participation rules

- Teams contain 1-4 registered participants.
- The core challenge implementation must be built during the 24-hour window.
- Open-source packages, pretrained models, and AI coding tools are allowed.
- Fabricated logs, hardcoded conversations presented as live, or copied work may lead to disqualification.
- External libraries, APIs, assets, and pre-existing code must be declared.
- No participant may manually write agent messages during the judged run.
- Teams are responsible for model access, rate limits, licenses, and API cost.

BEFORE YOU SUBMIT

Final participant checklist

Run through this list once on a clean setup. The judge should be able to understand the system without reading the source code first.

System

- Five agents load with separate goals and state.
- The judge can enter the resource pool and start the crisis.
- The transcript shows proposals, objections, offers, commitments, and votes.
- The first negotiation contains at least three rounds.
- Every department selects one unchanged mode package.
- Baseline risk is 24 or lower with no more than one Sacrifice mode.
- The sacrificing department receives two return commitments.
- Every candidate plan has a version number.
- The validator blocks unsafe plans.
- Commander approval requires PASS and four matching votes.
- The event makes affected plans stale and starts renegotiation.
- Logs and final plans export correctly.

Submission

- The README setup works from a clean environment.
- The run command and reset steps are easy to find.
- The repository contains no secrets or private data.
- The architecture diagram matches the implementation.
- Opening and post-event logs are included.
- The 3-5 minute video opens correctly.
- Models, APIs, libraries, assets, and reused code are declared.
- Team member names and roles are complete.

Quick answers

Question	Answer
Do we need a dataset?	No. The tables and event inputs in this guide are the full scenario.
Must each agent use a different model?	No. The roles and state must be separate; the model may be shared.
Is a web dashboard required?	Yes. A visual UI dashboard is mandatory for judging. Terminal or notebook interfaces will not be accepted.
Can we use an optimizer?	Yes. Show how it works with the agents and the deterministic validator.
Can we change the mode packages?	No. The published resource values and risk score of each mode are fixed.
What if the plan is impossible?	Return INFEASIBLE, explain why, and request a specific external change.

Your goal	Build a system in which the agents can disagree, make real tradeoffs, verify the numbers, and change the plan when the emergency changes.
------------------	---

HOST / LOOPLAB

TECHNOLOGY PARTNER / LIFIXLABS

ARES Accord | 24-Hour AI/ML Hackathon | Participant Challenge Guide | Version 2.1